

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
data = pd.read_csv(
    '/content/IMDB Dataset.csv',
    encoding='latin-1',
    engine='python',
    on_bad_lines='skip'
)
```

```
data.head()
data.shape
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38017 entries, 0 to 38016
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   review      38017 non-null   object
1   sentiment   38017 non-null   object
dtypes: object(2)
memory usage: 594.1+ KB
```

```
# Encode labels
data['sentiment'] = data['sentiment'].map({'positive': 1, 'negative': 0})

texts = data['review'].values
labels = data['sentiment'].values
```

```
from sklearn.model_selection import train_test_split

train_texts, test_texts, train_labels, test_labels = train_test_split(
    texts, labels, test_size=0.2, random_state=42
)
```

```
from tensorflow.keras.preprocessing.text import Tokenizer

max_words = 10000

tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(train_texts)

X_train = tokenizer.texts_to_matrix(train_texts, mode='binary')
X_test = tokenizer.texts_to_matrix(test_texts, mode='binary')

y_train = np.asarray(train_labels).astype('float32')
y_test = np.asarray(test_labels).astype('float32')
```

```
X_val = X_train[:10000]
partial_X_train = X_train[10000:]

y_val = y_train[:10000]
partial_y_train = y_train[10000:]
```

```
from tensorflow.keras import models, layers

model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

```
from tensorflow.keras import optimizers, losses, metrics

model.compile(
    optimizer=optimizers.RMSprop(learning_rate=0.001),
    loss=losses.binary_crossentropy,
    metrics=[metrics.binary_accuracy]
)
```

```
history = model.fit(
    partial_X_train,
    partial_y_train,
    epochs=20,
    batch_size=512,
    validation_data=(X_val, y_val)
)
```

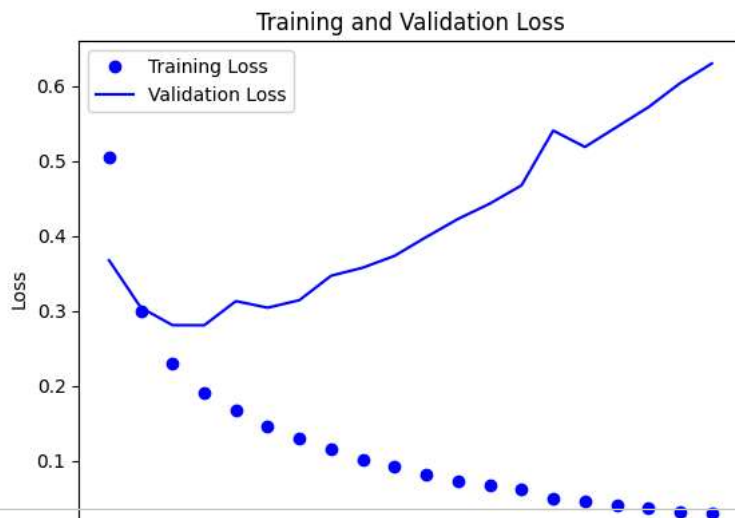
```
Epoch 1/20
40/40 ————— 4s 61ms/step - binary_accuracy: 0.7109 - loss: 0.5906 - val_binary_accuracy: 0.8742 - val_loss: 0.367
Epoch 2/20
40/40 ————— 1s 36ms/step - binary_accuracy: 0.8970 - loss: 0.3179 - val_binary_accuracy: 0.8802 - val_loss: 0.304
Epoch 3/20
40/40 ————— 1s 33ms/step - binary_accuracy: 0.9244 - loss: 0.2290 - val_binary_accuracy: 0.8906 - val_loss: 0.280
Epoch 4/20
40/40 ————— 1s 35ms/step - binary_accuracy: 0.9376 - loss: 0.1852 - val_binary_accuracy: 0.8911 - val_loss: 0.280
Epoch 5/20
40/40 ————— 1s 36ms/step - binary_accuracy: 0.9457 - loss: 0.1664 - val_binary_accuracy: 0.8796 - val_loss: 0.312
Epoch 6/20
40/40 ————— 1s 34ms/step - binary_accuracy: 0.9554 - loss: 0.1435 - val_binary_accuracy: 0.8890 - val_loss: 0.304
Epoch 7/20
40/40 ————— 2s 51ms/step - binary_accuracy: 0.9586 - loss: 0.1236 - val_binary_accuracy: 0.8888 - val_loss: 0.314
Epoch 8/20
40/40 ————— 2s 47ms/step - binary_accuracy: 0.9641 - loss: 0.1134 - val_binary_accuracy: 0.8801 - val_loss: 0.346
Epoch 9/20
40/40 ————— 2s 34ms/step - binary_accuracy: 0.9729 - loss: 0.0950 - val_binary_accuracy: 0.8828 - val_loss: 0.357
Epoch 10/20
40/40 ————— 3s 34ms/step - binary_accuracy: 0.9705 - loss: 0.0919 - val_binary_accuracy: 0.8827 - val_loss: 0.373
Epoch 11/20
40/40 ————— 1s 34ms/step - binary_accuracy: 0.9799 - loss: 0.0746 - val_binary_accuracy: 0.8781 - val_loss: 0.398
Epoch 12/20
40/40 ————— 1s 34ms/step - binary_accuracy: 0.9819 - loss: 0.0673 - val_binary_accuracy: 0.8777 - val_loss: 0.422
Epoch 13/20
40/40 ————— 1s 34ms/step - binary_accuracy: 0.9834 - loss: 0.0612 - val_binary_accuracy: 0.8760 - val_loss: 0.442
Epoch 14/20
40/40 ————— 2s 40ms/step - binary_accuracy: 0.9881 - loss: 0.0515 - val_binary_accuracy: 0.8748 - val_loss: 0.467
Epoch 15/20
40/40 ————— 2s 52ms/step - binary_accuracy: 0.9905 - loss: 0.0432 - val_binary_accuracy: 0.8650 - val_loss: 0.540
Epoch 16/20
40/40 ————— 1s 33ms/step - binary_accuracy: 0.9914 - loss: 0.0435 - val_binary_accuracy: 0.8723 - val_loss: 0.518
Epoch 17/20
40/40 ————— 1s 34ms/step - binary_accuracy: 0.9910 - loss: 0.0390 - val_binary_accuracy: 0.8708 - val_loss: 0.545
Epoch 18/20
40/40 ————— 3s 34ms/step - binary_accuracy: 0.9936 - loss: 0.0319 - val_binary_accuracy: 0.8708 - val_loss: 0.571
Epoch 19/20
40/40 ————— 1s 32ms/step - binary_accuracy: 0.9938 - loss: 0.0276 - val_binary_accuracy: 0.8677 - val_loss: 0.603
Epoch 20/20
40/40 ————— 1s 34ms/step - binary_accuracy: 0.9948 - loss: 0.0259 - val_binary_accuracy: 0.8689 - val_loss: 0.630
```

```
history_dict = history.history

loss_values = history_dict['loss']
val_loss_values = history_dict['val_loss']

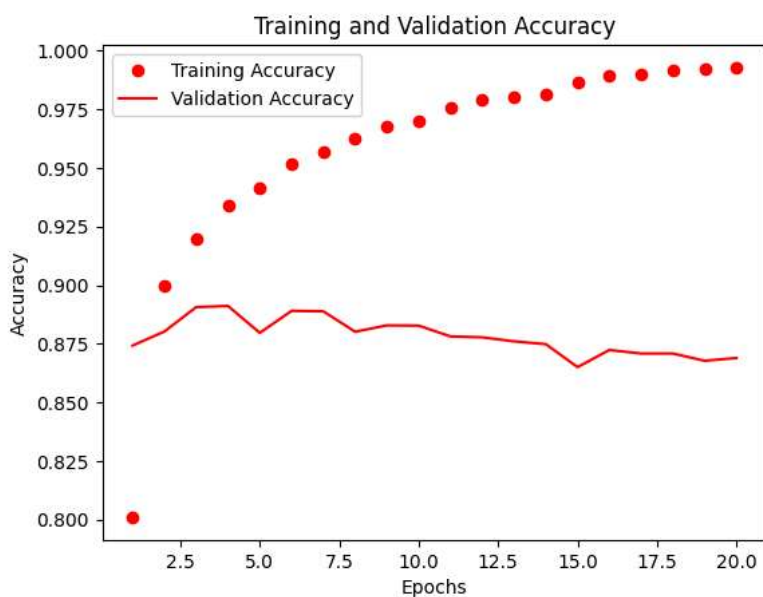
epochs = range(1, len(loss_values) + 1)

plt.plot(epochs, loss_values, 'bo', label='Training Loss')
plt.plot(epochs, val_loss_values, 'b', label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.show()
```



```
acc = history_dict['binary_accuracy']
val_acc = history_dict['val_binary_accuracy']

plt.plot(epochs, acc, 'ro', label='Training Accuracy')
plt.plot(epochs, val_acc, 'r', label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.show()
```



```
model.fit(
    partial_X_train,
    partial_y_train,
    epochs=3,
    batch_size=512,
    validation_data=(X_val, y_val)
)
```

```
Epoch 1/3
40/40 ————— 3s 71ms/step - binary_accuracy: 0.9966 - loss: 0.0199 - val_binary_accuracy: 0.8657 - val_loss: 0.679
Epoch 2/3
40/40 ————— 1s 33ms/step - binary_accuracy: 0.9957 - loss: 0.0215 - val_binary_accuracy: 0.8651 - val_loss: 0.704
Epoch 3/3
40/40 ————— 1s 34ms/step - binary_accuracy: 0.9960 - loss: 0.0204 - val_binary_accuracy: 0.8659 - val_loss: 0.720
<keras.src.callbacks.history.History at 0x7ce6a97cdc40>
```

```
result = model.predict(X_test)

y_pred = np.zeros(len(result))
for i, score in enumerate(result):
    y_pred[i] = 1 if score > 0.5 else 0
```

238/238  2s 7ms/step

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_pred, y_test)
print("Mean Absolute Error:", mae)
```

Mean Absolute Error: 0.13229879011046816

Start coding or [generate](#) with AI.